EC9600: Applied Algorithms

Dr. U. Thanuja

Senior Lecturer – Gr II

Dept. of Computer Engineering,

Faculty of Engineering,

University of Jaffna

thanuja@eng.jfn.ac.lk

Acknowledgement

Slides are taken from [1] and [2] and modified.

[1]. http://web.karabuk.edu.tr/hakankutucu/CME222/algorithm.ppt

[2]. https://www.cse.unr.edu/~bebis/CS477/Lect/InsertionSortBubbleSortSelectionSort.ppt

Introduction

- The methods of algorithm design form one of the core practical technologies of computer science.
- We start with a discussion of the algorithms needed to solve computational problems. The problem of *sorting* is used as a running example.
- We introduce a *pseudocode* to show how we shall specify the algorithms.

Algorithms

- The word *algorithm* comes from the name of a Persian mathematician Abu Ja'far Mohammed ibn-i Musa al Khowarizmi.

- In computer science, this word refers to a special method useable by a computer for solution of a problem. The statement of the problem specifies in general terms the desired input/output relationship.

- For example, sorting a given sequence of numbers into nondecreasing order provides fertile ground for introducing many standard design techniques and analysis tools.

The problem of sorting

Input: sequence $\langle a_1, a_2, \ldots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

Example:

Input: 8 2 4 9 3 6

Output: 2 3 4 6 8 9

Insertion Sort

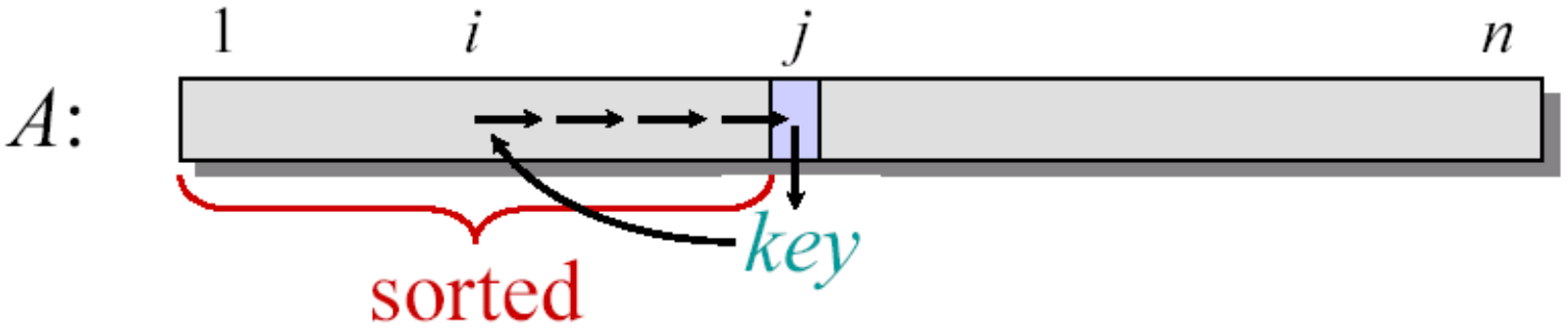
- Suitable for sorting small number of elements
- The Idea: like sorting a hand of playing cards
  - Start with an empty left hand and the cards facing down on the table.
  - Remove one card at a time from the table, and insert it into the correct position in the left hand
    - compare it with each of the cards already in the hand, from right to left
  - The cards held in the left hand are sorted
    - these cards were originally the top cards of the pile on the table

Figure 2.1: Sorting a hand of cards using insertion sort.

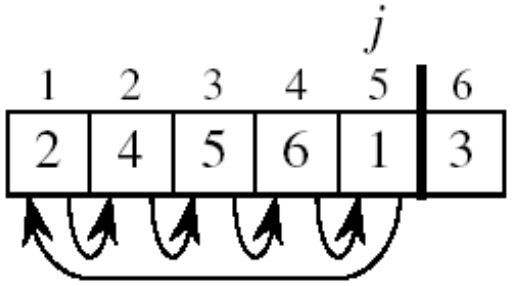
Insertion Sort

"pseudocode"

```
INSERTION-SORT (A, n)      ▷ A[1 . . n]
    for j ← 2 to n
        do key ← A[ j]
           i ← j – 1
           while i > 0 and A[i] > key
               do A[i+1] ← A[i]
                  i ← i – 1
           A[i+1] = key
```



Loop Invariant for Insertion Sort



Alg.: INSERTION-SORT*(A)*

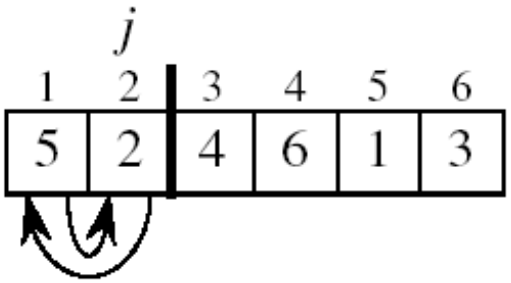
```
for j ← 2 to n
    do key ← A[ j ]
        Insert A[ j ] into the sorted sequence A[1 . . j -1]
        i ← j - 1
        while i > 0 and A[i] > key
            do A[i + 1] ← A[i]
                i ← i - 1
        A[i + 1] ← key
```

Invariant: at the start of the **for** loop the elements in $A[1 . . j-1]$ are in sorted order

Proving Loop Invariants

- Proving loop invariants works like induction
- **Initialization (base case):**
  - It is true prior to the first iteration of the loop
- **Maintenance (inductive step):**
  - If it is true before an iteration of the loop, it remains true before the next iteration
- **Termination:**
  - When the loop terminates, the invariant gives us a useful property that helps to show that the algorithm is correct
  - Stop the induction when the loop terminates

Loop Invariant for Insertion Sort

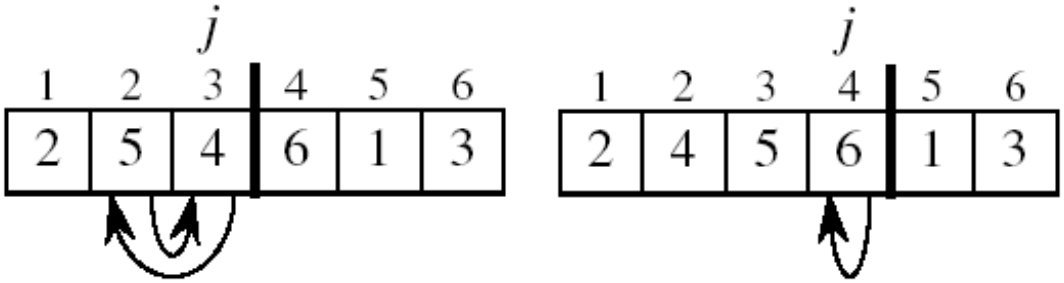


- **Initialization:**
  - Just before the first iteration, j = 2:

    the subarray $A[1 \ldots j-1] = A[1]$, (the element originally in $A[1]$) – is sorted

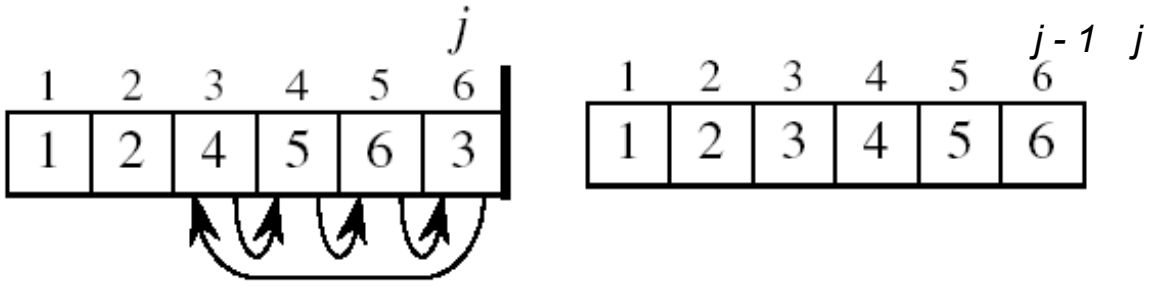
Loop Invariant for Insertion Sort

- **Maintenance:**
  - the **while** inner loop moves $A[j-1]$, $A[j-2]$, $A[j-3]$, and so on, by one position to the right until the proper position for key (which has the value that started out in $A[j]$) is found
  - At that point, the value of key is placed into this position.



Loop Invariant for Insertion Sort

- **Termination:**
  - The outer **for** loop ends when $j = n + 1 \Rightarrow j-1 = n$
  - Replace n with $j-1$ in the loop invariant:
    - the subarray $A[1 \ldots n]$ consists of the elements originally in $A[1 \ldots n]$, but in sorted order



- The entire array is sorted!

Invariant: at the start of the **for** loop the elements in $A[1 \ldots j-1]$ are in sorted order

Analysis of Algorithms

- **The theoretical study of computer-program performance and resource usage.**

- What's more important than performance?
    - modularity
    - correctness
    - maintainability
    - functionality
    - robustness
    - user-friendliness
    - programmer time
    - simplicity
    - extensibility
    - reliability

Analysis of Algorithms

Why study algorithms and performance?

- Algorithms help us to understand scalability.
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a ***language*** for talking about program behavior.
- The lessons of program performance generalize to other computing resources.
- Speed is fun!

Analysis of Algorithms

Efficiency of an algorithm is extremely important. It can be evaluated by analyzing time and space usage

- Time Complexity
- Space Complexity

Running Time

- The running time depends on the input: an already sorted sequence is easier to sort.
- Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.
- Generally, we seek upper bounds on the running time, because everybody likes a guarantee.

Running Time Analysis Types

Worst-case: (usually)

- T(n) = maximum time of algorithm on any input of size n.

Average-case: (sometimes)

- T(n) = expected time of algorithm over all inputs of size n.
- Need assumption of statistical distribution of inputs.

Best-case:

- Cheat with a slow algorithm that works fast on some input.

Machine-Independent time

The RAM Model

Machine independent algorithm design depends on a hypothetical computer called Random Acces Machine (RAM).

Assumptions:

- Each simple operation such as +, -, if ...etc takes exactly one time step.
- Loops and subroutines are not considered simple operations.
- Each memory acces takes exactly one time step.

Machine-Independent time

What is insertion sort's worst-case time?

- It depends on the speed of our computer,
  - relative speed (on the same machine),
  - absolute speed (on different machines).

BIG IDEA:

- Ignore machine-dependent constants.
- Look at **growth** of $T(n)$ as $n \to \infty$

"Asymptotic Analysis"

Machine-independent time: An example

A *pseudocode* for insertion sort (INSERTION SORT).

```
INSERTION-SORT(A)
1   for j ← 2 to length [A]
2         do key ← A[ j]
3         ∇ Insert A[j] into the sortted sequence A[1,..., j-1].
4          i ←  j – 1
5          while i > 0 and A[i] > key
6                   do A[i+1] ← A[i]
7                            i ← i – 1
8        A[i +1] ←  key
```

Analysis of INSERTION-SORT(contd.)

INSERTION-SORT(A)	cost	times
1 **for** $j \leftarrow 2$ **to** $length[A]$	c_1	n
2 **do** $key \leftarrow A[j]$	c_2	$n-1$
3 ∇ Insert $A[j]$ into the sorted sequence $A[1 \cdots j-1]$	0	$n-1$
4 $i \leftarrow j-1$	c_4	$n-1$
5 **while** $i > 0$ *and* $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 **do** $A[i+1] \leftarrow A[i]$	c_6	$\sum_{j=2}^{n} (t_j - 1)$
7 $i \leftarrow i-1$	c_7	$\sum_{j=2}^{n} (t_j - 1)$
8 $A[i+1] \leftarrow key$	c_8	$n-1$

Analysis of INSERTION-SORT(contd.)

The total running time is

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1)$$
$$+ c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

Best Case Analysis

"**while** i > 0 and A[i] > key"

- The array is already sorted
  - $A[i] \leq key$ upon the first time the **while** loop test is run (when $i = j - 1$)
  - $t_j = 1$
- $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) = (c_1 + c_2 + c_4 + c_5 + c_8)n + (c_2 + c_4 + c_5 + c_8)$

  $= an + b = \Theta(n)$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\sum_{j=2}^{n} t_j + c_6\sum_{j=2}^{n}(t_j - 1) + c_7\sum_{j=2}^{n}(t_j - 1) + c_8(n-1)$$

Worst Case Analysis

- The array is in reverse sorted order **"while** i > 0 and A[i] > key"
  - Always A[i] > key in **while** loop test
  - Have to compare key with all elements to the left of the j-th position $\Rightarrow$ compare with j-1 elements $\Rightarrow$ $t_j = j$

using $\sum_{j=1}^{n} j = \frac{n(n+1)}{2} \Rightarrow \sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1 \Rightarrow \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$ we have:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + c_6\frac{n(n-1)}{2} + c_7\frac{n(n-1)}{2} + c_8(n-1)$$

$$= an^2 + bn + c \quad \leftarrow \text{ a quadratic function of n}$$

- $T(n) = \Theta(n^2)$ $\leftarrow$ order of growth in n^2

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\sum_{j=2}^{n} t_j + c_6\sum_{j=2}^{n}(t_j - 1) + c_7\sum_{j=2}^{n}(t_j - 1) + c_8(n-1)$$

Comparisons and Exchanges in Insertion Sort

INSERTION-SORT(*A*)	cost	times
for j ← 2 **to** n	c_1	n
do key ← A[j]	c_2	n-1
Insert A[j] into the sorted sequence A[1 . . j -1]	0	n-1
i ← j - 1	c_4	n-1
while i > 0 and A[i] > key	c_5	$\sum_{j=2}^{n} t_j$
do A[i + 1] ← A[i]	c_6	$\sum_{j=2}^{n} (t_j - 1)$
i ← i – 1	c_7	$\sum_{j=2}^{n} (t_j - 1)$
A[i + 1] ← key	c_8	n-1

≈ $n^2/2$ comparisons

≈ $n^2/2$ exchanges

Insertion Sort - Summary

- Advantages
  - Good running time for “almost sorted” arrays $\Theta(n)$
- Disadvantages
  - $\Theta(n^2)$ running time in worst and average case
  - $\approx n^2/2$ comparisons and exchanges

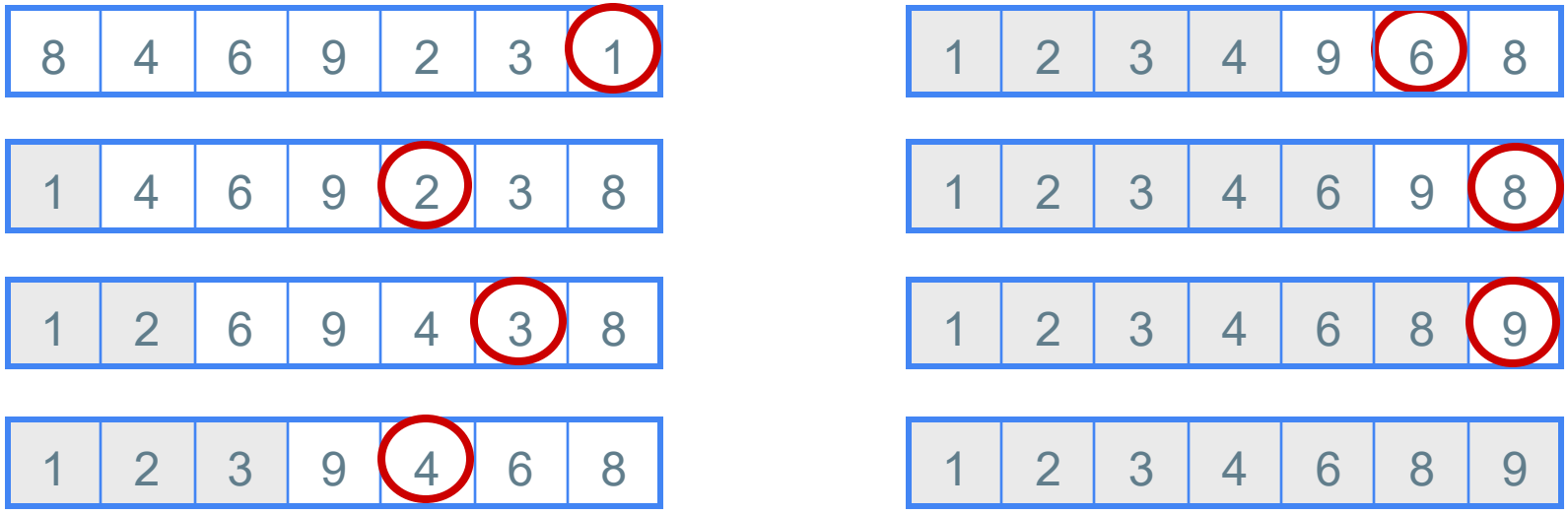
Selection Sort

- Idea:
  - Find the smallest element in the array
  - Exchange it with the element in the first position
  - Find the second smallest element and exchange it with the element in the second position
  - Continue until the array is sorted
- Disadvantage:
  - Running time depends only slightly on the amount of order in the file

Selection Sort

```
Alg.: SELECTION-SORT(A)
  n ← length[A]
  for j ← 1 to n - 1
        do smallest ← j
           for i ← j + 1 to n
                 do if A[i] < A[smallest]
                       then smallest ← i
           exchange A[j] ↔ A[smallest]
```

Example


8 4 6 9 2 3 1
1 4 6 9 2 3 8
1 2 6 9 4 3 8
1 2 3 9 4 6 8
1 2 3 4 9 6 8
1 2 3 4 6 9 8
1 2 3 4 6 8 9
1 2 3 4 6 8 9

Analysis of Selection Sort

Alg.: SELECTION-SORT(*A*)	cost	times
n ← length[A]	c_1	1
for j ← 1 **to** n - 1	c_2	n
do smallest ← j	c_3	n-1
for i ← j + 1 **to** n	c_4	$\sum_{j=1}^{n-1}(n-j+1)$
do if A[i] < A[smallest]	c_5	$\sum_{j=1}^{n-1}(n-j)$
then smallest ← i	c_6	$\sum_{j=1}^{n-1}(n-j)$
exchange A[j] ↔ A[smallest]	c_7	n-1

≈$n^2/2$ comparisons

≈n exchanges

$$T(n) = c_1 + c_2 n + c_3(n-1) + c_4\sum_{j=1}^{n-1}(n-j+1) + c_5\sum_{j=1}^{n-1}(n-j) + c_6\sum_{j=2}^{n-1}(n-j) + c_7(n-1) = \Theta(n^2)$$

Growth of Functions

- Although we can sometimes determine the exact running time of an algorithm, the extra precision is not usually worth the effort of computing it.
- For large inputs, the multiplicative constants and lower order terms of an exact running time are dominated by the effects of the input size itself.

Asymptotic Notation

The notation we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of natural numbers.

$$\mathbf{N} = \{0, 1, 2, ...\}$$

O Notation

- For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$$O(g(n)) = \left\{ \begin{array}{c} f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ s.t.} \\ 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0 \end{array} \right\}$$

- We use *O*-notation to give an asymptotic upper bound of a function, to within a constant factor.
- $f(n) = O(g(n))$ means that there existes some constant c s.t. $f(n)$ is always $\le cg(n)$ for large enough n.

Ω-Omega Notation

- For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$$\Omega(g(n)) = \left\{ \begin{array}{c} f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ s.t.} \\ 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0 \end{array} \right\}$$

- We use *Ω*-notation to give an asymptotic lower bound on a function, to within a constant factor.
- $f(n) = \Omega(g(n))$ means that there exists some constant *c* s.t. $f(n)$ is always $\ge cg(n)$ for large enough *n*.

Θ -Theta notation

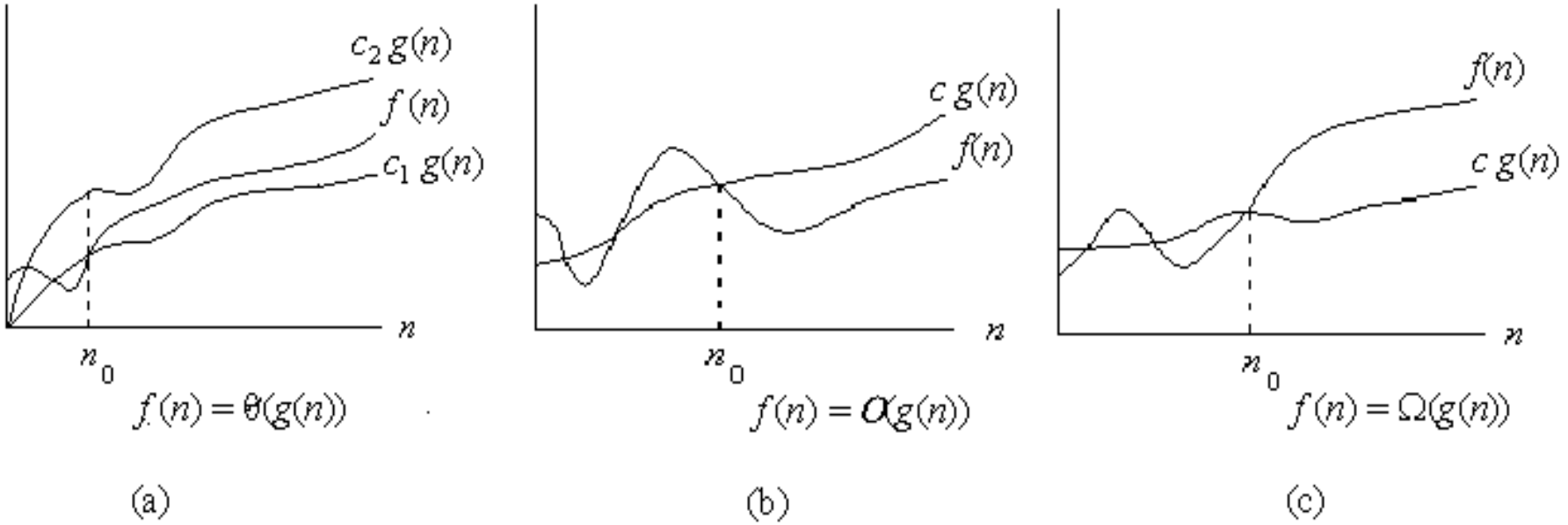
- For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions

$$\Theta(g(n)) = \left\{ \begin{array}{c} f(n) : \text{there exist positive constants } c_1, c_2, \text{and } n_0 \text{ s.t.} \\ 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0 \end{array} \right\}$$

- A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be "sand- wiched" between $c_1 g(n)$ and $c_2 g(n)$ or sufficienly large *n.*

- $f(n) = \Theta(g(n))$ means that there exists some constant c_1 and c_2 s.t. $c_1 g(n) \le f(n) \le c_2 g(n)$ for large enough *n*.

Asymptotic notation



Graphic examples of Θ, O, and Ω.

Example 1

Show that $f(n) = \frac{1}{2}n^2 - 3n = \Theta(n^2)$

We must find c_1 and c_2 such that

$$c_1 n^2 \le \frac{1}{2}n^2 - 3n \le c_2 n^2$$

Dividing bothsides by n^2 yields

$$c_1 \le \frac{1}{2} - \frac{3}{n} \le c_2$$

For $n_0 \ge 7$, $\frac{1}{2}n^2 - 3n = \Theta(n^2)$

Theorem

For any two functions f(n) and g(n), we have $f(n) = \Theta(g(n))$ if and only if

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

Example 2

$$f(n) = 3n^2 - 2n + 5 = \Theta(n^2)$$

Because :

$$3n^2 - 2n + 5 = \Omega(n^2)$$

$$3n^2 - 2n + 5 = O(n^2)$$

Example 3

$$3n^2 - 100n + 6 = O(n^2) \quad \text{since for } c = 3,\ 3n^2 > 3n^2 - 100n + 6$$

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3,\ 3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1,\ n^3 > 3n^2 - 100n + 6$ when $n > 3$

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3,\ 3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1,\ n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any $c,\ cn < 3n^2$ when $n > c$

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3,\ 3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1,\ n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any $c,\ cn < 3n^2$ when $n > c$

$3n^2 - 100n + 6 = \Omega(n^2)$ since for $c = 2,\ 2n^2 < 3n^2 - 100n + 6$ when $n > 100$

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3,\ 3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1,\ n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any $c,\ cn < 3n^2$ when $n > c$

$3n^2 - 100n + 6 = \Omega(n^2)$ since for $c = 2,\ 2n^2 < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 \neq \Omega(n^3)$ since for $c = 3,\ 3n^2 - 100n + 6 < n^3$ when $n > 3$

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3$, $3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1$, $n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any c, $cn < 3n^2$ when $n > c$

$3n^2 - 100n + 6 = \Omega(n^2)$ since for $c = 2$, $2n^2 < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 \neq \Omega(n^3)$ since for $c = 3$, $3n^2 - 100n + 6 < n^3$ when $n > 3$

$3n^2 - 100n + 6 = \Omega(n)$ since for any c, $cn < 3n^2 - 100n + 6$ when $n > 100$

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3$, $3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1$, $n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any c, $cn < 3n^2$ when $n > c$

$3n^2 - 100n + 6 = \Omega(n^2)$ since for $c = 2$, $2n^2 < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 \neq \Omega(n^3)$ since for $c = 3$, $3n^2 - 100n + 6 < n^3$ when $n > 3$

$3n^2 - 100n + 6 = \Omega(n)$ since for any c, $cn < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 = \Theta(n^2)$ since both O and Ω apply.

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3$, $3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1$, $n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any c, $cn < 3n^2$ when $n > c$

$3n^2 - 100n + 6 = \Omega(n^2)$ since for $c = 2$, $2n^2 < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 \neq \Omega(n^3)$ since for $c = 3$, $3n^2 - 100n + 6 < n^3$ when $n > 3$

$3n^2 - 100n + 6 = \Omega(n)$ since for any c, $cn < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 = \Theta(n^2)$ since both O and Ω apply.

$3n^2 - 100n + 6 \neq \Theta(n^3)$ since only O applies.

Example 3

$3n^2 - 100n + 6 = O(n^2)$ since for $c = 3$, $3n^2 > 3n^2 - 100n + 6$

$3n^2 - 100n + 6 = O(n^3)$ since for $c = 1$, $n^3 > 3n^2 - 100n + 6$ when $n > 3$

$3n^2 - 100n + 6 \neq O(n)$ since for any c, $cn < 3n^2$ when $n > c$

$3n^2 - 100n + 6 = \Omega(n^2)$ since for $c = 2$, $2n^2 < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 \neq \Omega(n^3)$ since for $c = 3$, $3n^2 - 100n + 6 < n^3$ when $n > 3$

$3n^2 - 100n + 6 = \Omega(n)$ since for any c, $cn < 3n^2 - 100n + 6$ when $n > 100$

$3n^2 - 100n + 6 = \Theta(n^2)$ since both O and Ω apply.

$3n^2 - 100n + 6 \neq \Theta(n^3)$ since only O applies.

$3n^2 - 100n + 6 \neq \Theta(n)$ since only Ω applies.

Space Complexity

- Space complexity is a measure of the amount of working storage an algorithm needs
  - how much memory, in the worst case, is needed at any point in the algorithm
- Example:

1.
```
int sum(int x, int y, int z) {
    int r = x + y + z;
    return r;
}
```
2.
```
int sum(int a[], int n) {
    int r = 0;
    for (int i = 0; i < n; ++i) {
        r += a[i];
    }
    return r;
}
```

Space Complexity

3.
```
void matrixAdd(int a[], int b[], int c[], int n) {
        for (int i = 0; i < n; ++i) {
                c[i] = a[i] + b[j]
        }
}
```
4.
```
void matrixMultiply(int a[], int b[], int c[][], int n) { // not legal C++
        for (int i = 0; i < n; ++i) {
                for (int j = 0; j < n; ++j) {
                        c[i] = a[i] + b[j];
                }
        }
}
```

Space Complexity

Algorithm	Space Complexity (Worst Case)
Insertion Sort	O(1)
Bubble Sort	O(1)
Selection Sort	O(1)

Standard notations and common functions

Standard notations and common functions

- Floors and ceilings

$$x-1<\lfloor x\rfloor\le x\le\lceil x\rceil<x+1$$

Standard notations and common functions

- Logarithms:

$$\lg n = \log_2 n$$

$$\ln n = \log_e n$$

$$\log^k n = (\log n)^k$$

$$\lg \lg n = \lg(\lg n)$$

Standard notations and common functions

- Logarithms:

  For all real $a>0$, $b>0$, $c>0$, and n

$$a = b^{\log_b a}$$

$$\log_c (ab) = \log_c a + \log_c b$$

$$\log_b a^n = n \log_b a$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

Standard notations and common functions

- Logarithms:

$$\log_b(1/a) = -\log_b a$$

$$a^{\log_b c} = c^{\log_b a}$$

$$\log_b a = \frac{1}{\log_a b}$$

Standard notations and common functions

- Factorials

  For $n \geq 0$ the Stirling approximation:

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1+\Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n)$$

$$n! = \omega(2^n)$$

$$\lg(n!) = \Theta(n \lg n)$$

Standard notations and common functions

- Constant Series: For integers a and b, $a \le b$,

$$\sum_{i=a}^{b} 1 = b - a + 1$$

- Linear Series (Arithmetic Series): For $n \ge 0$,

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- Quadratic Series: For $n \ge 0$,

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Standard notations and common functions

- Cubic Series: For $n \geq 0$,

$$\sum_{i=1}^{n} i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

- Geometric Series: For real $x \neq 1$,

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

For $|x| < 1$, $\displaystyle\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$

Standard notations and common functions

- Linear-Geometric Series: For $n \geq 0$, real $c \neq 1$,

$$\sum_{i=1}^{n} ic^i = c + 2c^2 + \cdots + nc^n = \frac{-(n+1)c^{n+1} + nc^{n+2} + c}{(c-1)^2}$$

- Harmonic Series: nth harmonic number, $n \in \mathbb{I}^+$,

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

$$= \sum_{k=1}^{n} \frac{1}{k} = \ln(n) + O(1)$$

Standard notations and common functions

- Telescoping Series:

$$\sum_{k=1}^{n} a_k - a_{k-1} = a_n - a_0$$

- Geometric Series: For $|x| < 1$,

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$$